

embarcadero®

EMBARCADERO CONFERENCE 2025

A JORNADA DO HERÓI | 30 ANOS DE INOVAÇÃO



| Fáblio Ribeiro

Tech Lead e Arquiteto Delphi no BDMG

Pós Graduação em Arquitetura de Software pela PUC Minas

+13 anos de experiência em Delphi, focado em modernização

7 anos com foco em APIs com Delphi

APIs REST com Delphi:

Boas Práticas, Arquitetura Limpa e Antipadrões no Mundo Real

Fábio Ribeiro

"Um mergulho nos erros mais comuns e nas soluções que aumentam a qualidade, segurança e manutenibilidade das APIs — porque mais do que escrever código, é a forma como estruturamos a API que define seu sucesso."

| O que vamos ver:

- REST vs RESTful
- Boas Práticas na Modelagem REST
- Status Code & Tratamento de Erros
- Antipadrões Estruturais
- Performance e Eficiência

REST vs RESTful

| Entendendo a diferença de forma prática

REST

- Estilo arquitetural proposto por Roy Fielding (2000)
- Define **6** princípios:
 - Client-Server
 - Stateless
 - Cacheable
 - Interface Uniforme (inclui HATEOAS)
 - Layered System
 - (Opcional) Code on Demand

RESTful

- Aplicação correta dos princípios REST em uma API
- Respeita semântica de métodos HTTP, URIs claras, stateless, etc
- Relacionado ao **Modelo de Maturidade de Richardson**:
 - **Nível 0**: tudo em um único endpoint
 - **Nível 1**: uso de recursos (URIs específicas)
 - **Nível 2**: métodos HTTP corretos
 - **Nível 3**: HATEOAS (hipermídia)

HATEOAS

Hypermedia As The Engine Of Application State

- Define links para ações possíveis na resposta da API
- Recomendado no REST, mas raramente implementado
- Em APIs internas, muitas vezes é desnecessário e complexo

```
{
  "id": 101,
  "nome": "Notebook Dell Inspiron",
  "preco": 3999.90,
  "estoque": 12,
  "_links": {
    "inativar": {
      "href": "/produtos/101/inativar",
      "method": "PATCH"
    },
    "deletar": {
      "href": "/produtos/101",
      "method": "DELETE"
    }
  }
}
```

Boas Práticas na Modelagem REST: URI, Métodos e Payloads

URIs claras, métodos corretos e payloads enxutos evitam problemas futuros

| Como criar URIs para sua API?

1. Tenha recursos claros e métodos corretos

- Prefira nomes que descrevam bem o recurso.

✅ Exemplo: */clientes* e */pedidos*

- Utilize os verbos http corretamente

❌ Pare de usar POST para tudo!

| Como criar URIs para sua API?

2. Use substantivos no plural

- Representam coleções de recursos.
- Exemplos:

 ***/cliente e /pedido/123***

 ***/clientes e /pedidos/123***

| Como criar URIs para sua API?

3. Mantenha a consistência

- Padrões iguais em toda a API.
- Recursos compostos, prefira **hífen (-)** como separador. (kebab-case)
- Exemplos:

✗ */clientes* e */cliente/123/historico_compra*

✓ */clientes* e */clientes/123/historico-compras*

| Como criar URIs para sua API?

4. Vai ter verbo na URI sim (De forma adequada)

- Prefira o verbo HTTP sempre que possível:

DELETE **/clientes/1** ✓

PATCH **/clientes/1** com { "ativo": false } ✓

- Quando a operação não for CRUD puro, a ação na URI é válida.

PATCH **/produtos/1/inativar** ✓

PATCH **/produtos/1/alterarStatus**

✗

POST **/pedidos/1/cancelar** ✓

POST **/cancelarPedido/1** ✗

| Como criar URIs para sua API?

5. Use hierarquia para sub-recursos

- Relacione entidades de forma natural.

/pedidos/123/itens/45/descontos ✓

/pedidos/123/descontos/itens/45 ✗

- Evite URIs profundas de mais

Ex.: ***/a/b/c/d/e/f*** ✗

| Como criar URIs para sua API?

6. Nunca exponha dados sensíveis na URI

✗ /usuarios/123?senha=abc

✗ /clientes?cpf=999999999999

✓ Dados sensíveis devem ir no **payload**.
{"cpf":"999999999999"}

| Como criar URIs para sua API?

7. Payloads devem ser objetivos

✗ Evite campos desnecessários ou estruturas aninhadas demais

✓ Se precisar ser grande, ofereça opções resumidas (Sparse Fieldssets ou Projection)

- `fields=id,name,email` → Campos específicos
- `view=simple` → apenas os campos essenciais
- `view=full` → Todos os campos disponíveis

Status Code & Tratamento de Erros

| Devolva erro com dignidade: status certo e mensagem clara

| Erros bem tratados = APIs mais confiáveis

✓ Utilize os status code corretos

- 2xx – Sucesso
- 3xx - Redirecionamento
- 4xx – Erro do cliente
- 5xx – Erro do servidor

✗ Usar status 500 para qualquer erro

✗ `raise Exception.Create` → não use para regra de negócio

✓ Use `EHorseException` para status e mensagens claras

Antipadrões Estruturais

| Antipadrões de estrutura que destroem APIs

| Antipadrões de estrutura que destroem APIs

- ✗ Lógica de negócio no controller
- ✗ DAO chamando outro DAO
- ✗ Entidade sendo usada como DTO (Utilize DTO para E/S)
- ✗ Falta de padrão de organização → manutenção difícil

Não importa o padrão escolhido, o importante é **ter um** e segui-lo.

- ✓ Use arquitetura limpa (Controller → Service → Repository)
- ✓ Use interfaces e factories

Performance e Eficiência

| Cada milissegundo conta — principalmente em escala

| Cada milissegundo conta

- Reduza o tamanho dos payloads ✓
Ofereça ?resumido=true ou DTOs leves
- Evite middlewares pesados ✓
Faça autenticação, mas sem consulta de banco em toda requisição
- Use cache inteligente sempre que possível ✓
- Preocupe-se com performance antes do primeiro endpoint ✓

Conclusão

| APIs REST de verdade são pensadas, não improvisadas

| APIs REST de verdade são pensadas, não improvisadas

- REST não é sobre modinha — é sobre clareza e consistência ✓
- RESTful é o que você faz, não só o que você diz ✓
- Arquitetura limpa ajuda no longo prazo ✓
- Status code certo, rota clara, payload enxuto = API saudável ✓
- Faça pelo você do futuro ou faça por quem vai dar manutenção ✓

embarcadero®

EMBARCADERO CONFERENCE 2025

A JORNADA DO HERÓI | 30 ANOS DE INOVAÇÃO



Quer me ver na
#ECON26?

Leia o QRCode **pelo APP** e
avalie minha palestra!



Fábio Ribeiro



<https://www.linkedin.com/in/fhmr-tech/>



Fabiohenriquebhz@gmail.com



(31) 9 9326-1135